

# HW1 Econ144

Mason Lee

2025-10-08

```
library(readr)
library(lattice)
library(foreign)
library(MASS)
library(car)
```

```
## Loading required package: carData
```

```
require(stats)
require(stats4)
```

```
## Loading required package: stats4
```

```
library(KernSmooth)
```

```
## KernSmooth 2.23 loaded
## Copyright M. P. Wand 1997-2009
```

```
library(fastICA)
library(cluster)
library(leaps)
library(mgcv)
```

```
## Loading required package: nlme
```

```
## This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
```

```
library(rpart)
library(pan)
library(mgcv)
library(DAAG)
```

```
##
## Attaching package: 'DAAG'
```

```
## The following object is masked from 'package:car':
##
##      vif
```

```
## The following object is masked from 'package:MASS':  
##  
## hills
```

```
library("TTR")  
library(tis)
```

```
##  
## Attaching package: 'tis'
```

```
## The following object is masked from 'package:TTR':  
##  
## lags
```

```
## The following object is masked from 'package:mgcv':  
##  
## ti
```

```
require("datasets")  
require(graphics)  
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':  
## method from  
## as.zoo.data.frame zoo
```

```
##  
## Attaching package: 'forecast'
```

```
## The following object is masked from 'package:tis':  
##  
## easter
```

```
## The following object is masked from 'package:nlme':  
##  
## getResponse
```

```
require(astsa)
```

```
## Loading required package: astsa
```

```
##  
## Attaching package: 'astsa'
```

```
## The following object is masked from 'package:forecast':  
##  
## gas
```

```
library(RColorBrewer)
library(plotrix)
library(tsibble)
```

```
## Registered S3 method overwritten by 'tsibble':
##   method          from
##   as_tibble.grouped_df dplyr
```

```
##
## Attaching package: 'tsibble'
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, union
```

```
library(ggplot2)
library(tidyr)
library(USgas)
library(fpp3)
```

```
## -- Attaching packages ----- fpp3 1.0.2 --
```

```
## v tibble      3.2.1      v tsibbledata 0.4.1
## v dplyr       1.1.4      v feasts      0.4.2
## v lubridate   1.9.4      v fable       0.4.1
```

```
## -- Conflicts ----- fpp3_conflicts --
```

```
## x feasts::ACF()          masks nlme::ACF()
## x dplyr::between()       masks tis::between()
## x dplyr::collapse()      masks nlme::collapse()
## x lubridate::date()      masks base::date()
## x lubridate::day()       masks tis::day()
## x dplyr::filter()        masks stats::filter()
## x lubridate::hms()       masks tis::hms()
## x tsibble::intersect()   masks base::intersect()
## x lubridate::interval()  masks tsibble::interval()
## x dplyr::lag()           masks stats::lag()
## x lubridate::month()     masks tis::month()
## x lubridate::period()    masks tis::period()
## x lubridate::POSIXct()   masks tis::POSIXct()
## x lubridate::quarter()   masks tis::quarter()
## x dplyr::recode()        masks car::recode()
## x dplyr::select()        masks MASS::select()
## x tsibble::setdiff()     masks base::setdiff()
## x lubridate::today()     masks tis::today()
## x tsibble::union()       masks base::union()
## x lubridate::year()      masks tis::year()
## x lubridate::ymd()       masks tis::ymd()
```

```
q1 <- read_csv("q1data.csv")
```

```
## Rows: 640 Columns: 4
```

```
## -- Column specification -----  
## Delimiter: ","  
## chr (1): date  
## dbl (3): rpce, rdpi, date_num  
##  
## i Use 'spec()' to retrieve the full column specification for this data.  
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
tute1 <- read_csv("tute1.csv")
```

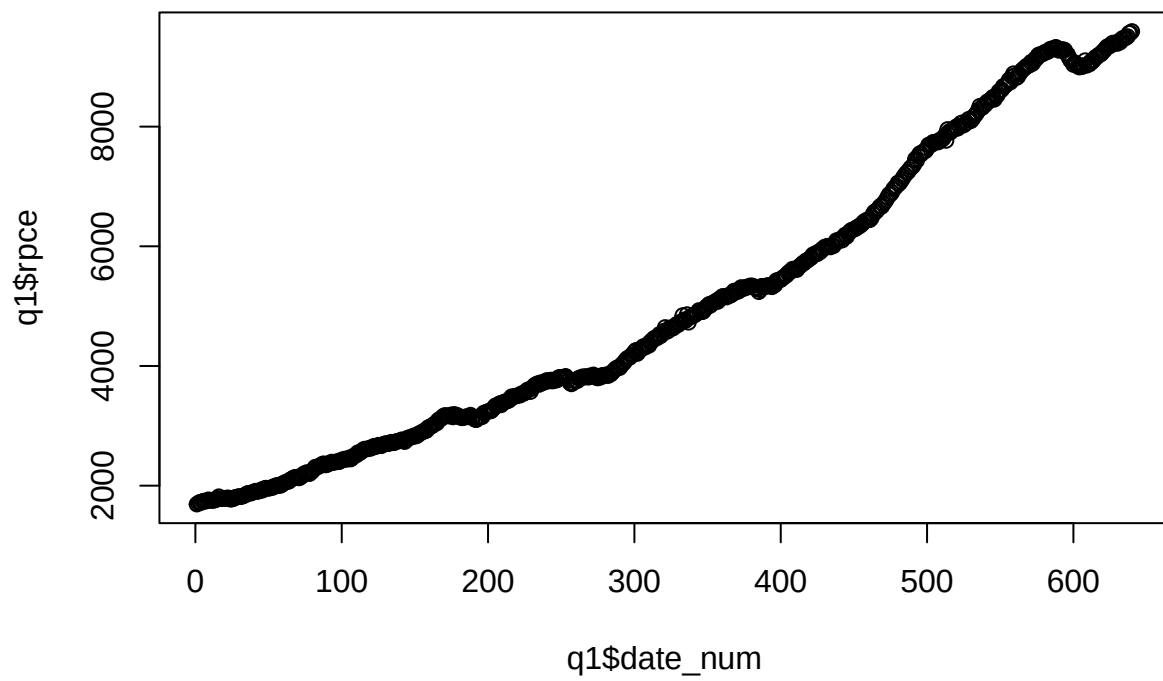
```
## Rows: 100 Columns: 4  
## -- Column specification -----  
## Delimiter: ","  
## dbl (3): Sales, AdBudget, GDP  
## date (1): Quarter  
##  
## i Use 'spec()' to retrieve the full column specification for this data.  
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

## Textbook A

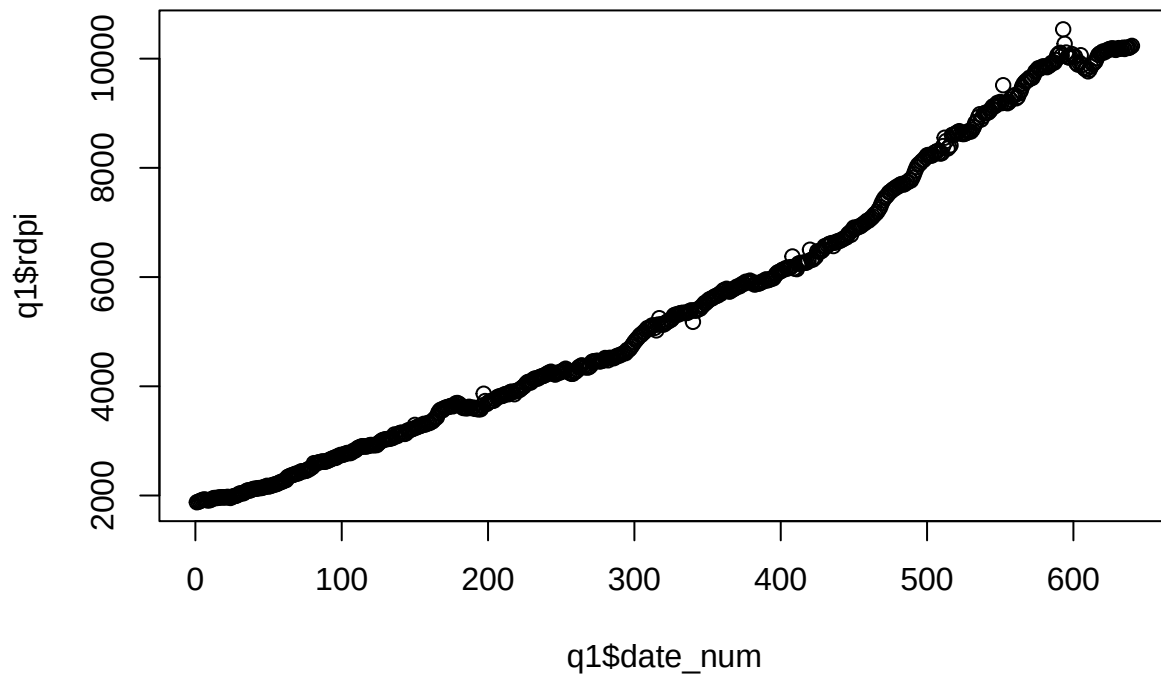
### 3.1

a)

```
plot(q1$date_num, q1$rpce)
```



```
plot(q1$date_num, q1$rdpi)
```



```
g_rpce <- 100 * diff(log(q1$rpce))
g_rpdi <- 100 * diff(log(q1$rdpi))

mean(g_rpce)
```

```
## [1] 0.2717571
```

```
sd(g_rpce) #for volatility
```

```
## [1] 0.546044
```

```
mean(g_rpdi)
```

```
## [1] 0.2654316
```

```
sd(g_rpdi) #for volatility
```

```
## [1] 0.7209097
```

The mean month over month growth rates of rpce and rdpi are very similar, but rdpi is much more volatile. This aligns well with the permanent income model, which states that consumption growth is less volatile than income growth. Consumption doesn't react much to short term changes in income.

b)

```
summary(lm(g_rpce ~ g_rpdj))

##
## Call:
## lm(formula = g_rpce ~ g_rpdj)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.03967 -0.29727  0.01525  0.30417  2.44545
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.22542     0.02242  10.055 < 2e-16 ***
## g_rpdj       0.17457     0.02920   5.978 3.77e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5318 on 637 degrees of freedom
## Multiple R-squared:  0.05312,    Adjusted R-squared:  0.05163
## F-statistic: 35.73 on 1 and 637 DF,  p-value: 3.768e-09
```

The linear regression formula ( $g\_rpce = 0.22542 + 0.17457 \cdot g\_rdpi$ ) tells us that for every increase of 1 in  $g\_rdpi$ ,  $rpce$  goes up by 0.17457. The R squared value of 0.05163 tells us that the linear relationship is weak, and that there is almost no correlation in growth rates.

c)

```
gy_l1 <- c(NA, head(g_rpdj, -1))

df <- na.omit(data.frame(g_rpce, g_rpdj, gy_l1))

m2 <- lm(g_rpce ~ g_rpdj + gy_l1, data = df)
summary(m2)

##
## Call:
## lm(formula = g_rpce ~ g_rpdj + gy_l1, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.00734 -0.28702 -0.00006  0.29797  2.55131
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.19887     0.02405   8.268 7.98e-16 ***
## g_rpdj       0.18727     0.02939   6.372 3.58e-10 ***
## gy_l1        0.08286     0.02939   2.820  0.00496 **
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5285 on 635 degrees of freedom
## Multiple R-squared:  0.06479,    Adjusted R-squared:  0.06185
## F-statistic:    22 on 2 and 635 DF,  p-value: 5.8e-10
```

After adding in the lag, we see that both the growth rate and the lag of income affect the growth rate in personal consumption. They are both significant variables. In this new regression, R squared went up to 0.06. It is still a bad fit, but it did increase slightly.

### 3.5

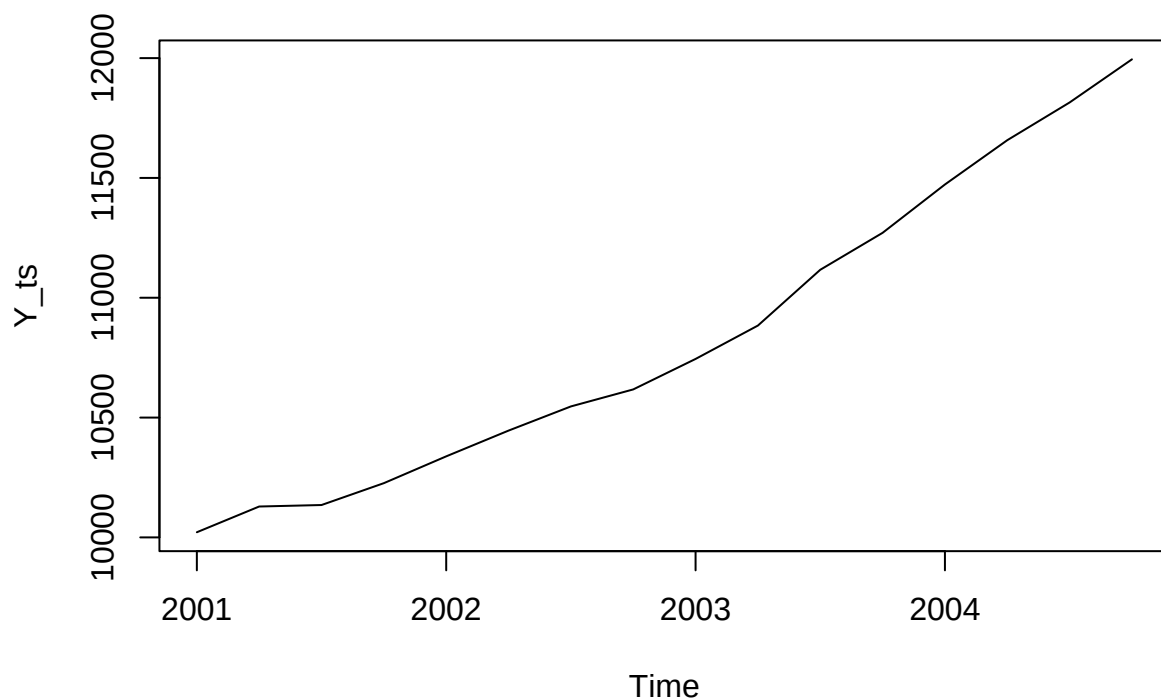
a)

```
date <- as.Date(c(
  "2001-01-01", "2001-04-01", "2001-07-01", "2001-10-01",
  "2002-01-01", "2002-04-01", "2002-07-01", "2002-10-01",
  "2003-01-01", "2003-04-01", "2003-07-01", "2003-10-01",
  "2004-01-01", "2004-04-01", "2004-07-01", "2004-10-01"
))

Y <- c(10021.5, 10128.9, 10135.1, 10226.3,
       10338.2, 10445.7, 10546.5, 10617.5,
       10744.6, 10884.0, 11116.7, 11270.9,
       11472.6, 11657.5, 11814.9, 11994.8)

Y_ts <- ts(Y, start = c(2001, 1), frequency = 4)
plot(Y_ts)
```





Y is not weakly stationary due to the uptick. The mean is not constant.

b)

```
g1 <- 100 * diff(Y) / head(Y, -1)
g1_ts <- ts(g1, start = c(2001, 2), frequency = 4)

head(g1)
```

```
## [1] 1.07169585 0.06121099 0.89984312 1.09423741 1.03983285 0.96499038
```

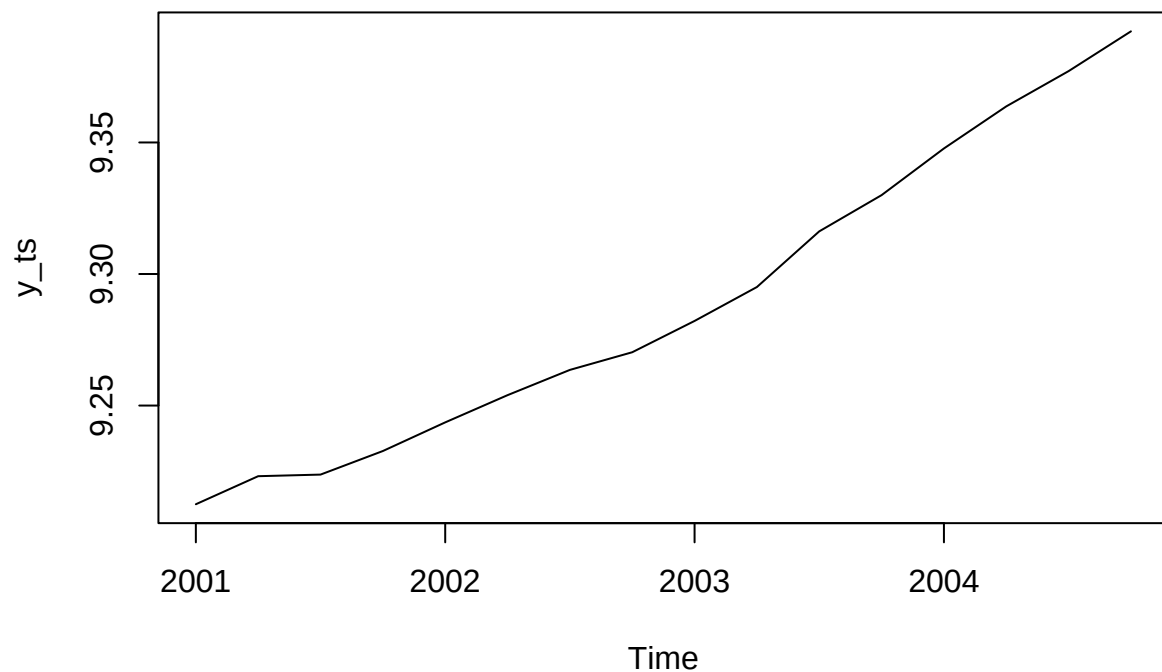
```
head(g1_ts)
```

```
##           Qtr1           Qtr2           Qtr3           Qtr4
## 2001           1.07169585 0.06121099 0.89984312
## 2002 1.09423741 1.03983285 0.96499038
```

c)

```
y <- log(Y)
y_ts <- ts(y, start = c(2001,1), frequency = 4)

plot(y_ts)
```



This new plot is smoother and more linear than the non-logarithmic version. It is still not weakly stationary due to it trending upwards.

d)

```
g2 <- 100 * diff(y)
g2_ts <- ts(g2, start = c(2001,2), frequency = 4)
head(g2)
```

```
## [1] 1.06599390 0.06119226 0.89581866 1.08829395 1.03446378 0.96036409
```

```
head(g2_ts)
```

```
##           Qtr1           Qtr2           Qtr3           Qtr4
## 2001           1.06599390 0.06119226 0.89581866
## 2002 1.08829395 1.03446378 0.96036409
```

e)

```
comp1 <- cbind(g1, g2)
comp1
```

```
##           g1           g2
## [1,] 1.07169585 1.06599390
## [2,] 0.06121099 0.06119226
## [3,] 0.89984312 0.89581866
## [4,] 1.09423741 1.08829395
## [5,] 1.03983285 1.03446378
## [6,] 0.96499038 0.96036409
## [7,] 0.67320912 0.67095319
## [8,] 1.19708029 1.18997196
## [9,] 1.29739590 1.28905181
## [10,] 2.13800074 2.11546613
## [11,] 1.38710229 1.37757007
## [12,] 1.78956428 1.77374009
## [13,] 1.61166606 1.59881660
## [14,] 1.35020373 1.34116971
## [15,] 1.52265360 1.51117758
```

The percent change (g1) and the log growth (g2) figures are almost identical. This echoes what we noticed in the graphs (part a and c), which were very similar but slightly different in the middle.

### 3.6

a)

```
samp_moment <- mean(g2)
var_g2 <- var(g2)

samp_moment
```

```
## [1] 1.19827
```

```
var_g2
```

```
## [1] 0.233368
```

b)

```
rho <- acf(g2, plot = FALSE, lag.max = 4)$acf[2:5]
rho
```

```
## [1] 0.4275882 0.3358029 0.1379455 0.1052045
```

```

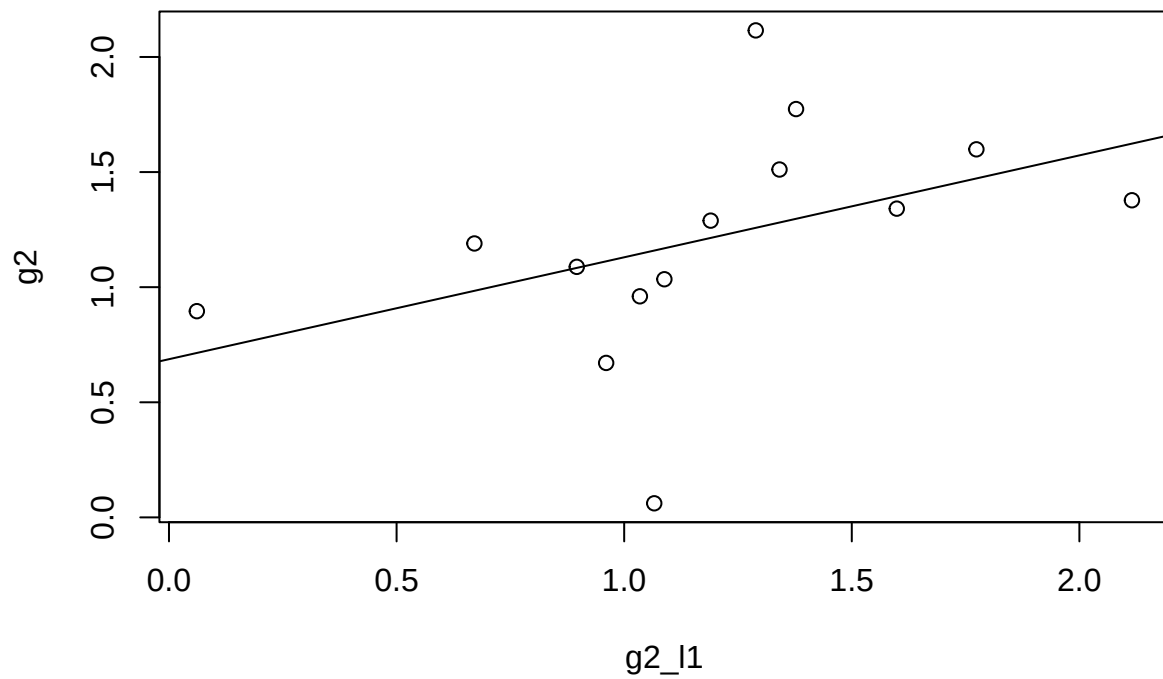
g2_l1 <- c(NA, head(g2, -1))
g2_l2 <- c(NA, NA, head(g2, -2))
g2_l3 <- c(NA, NA, NA, head(g2, -3))
g2_l4 <- c(NA, NA, NA, NA, head(g2, -4))

```

```

plot(g2_l1, g2); abline(lm(g2 ~ g2_l1))

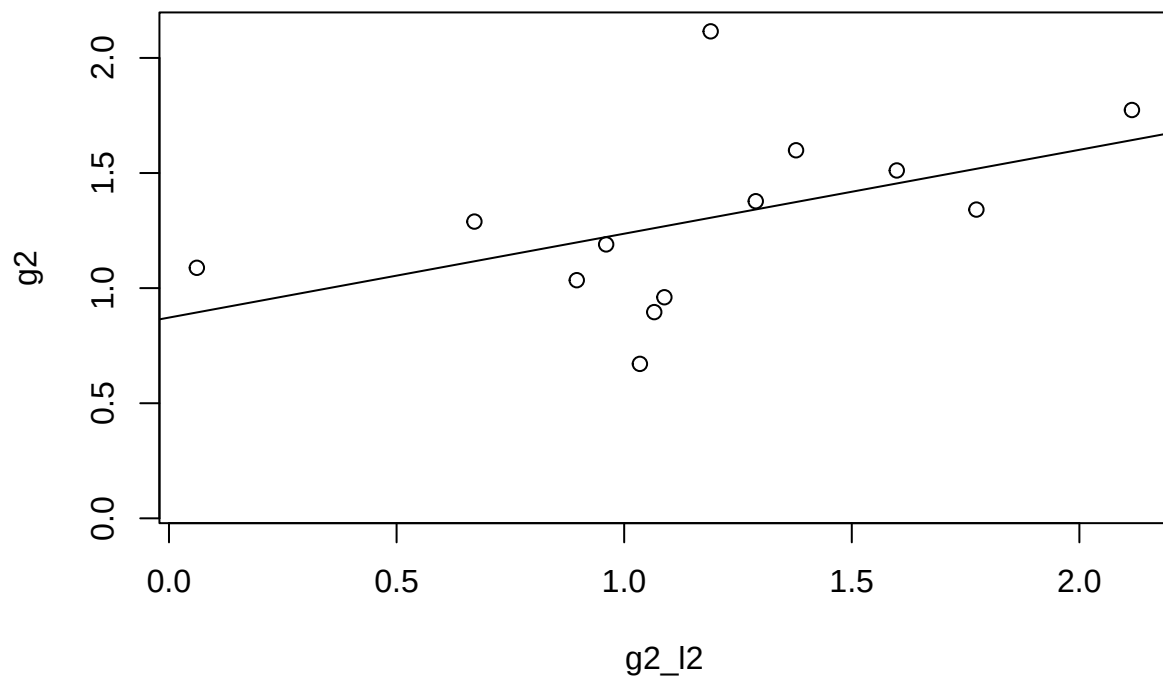
```



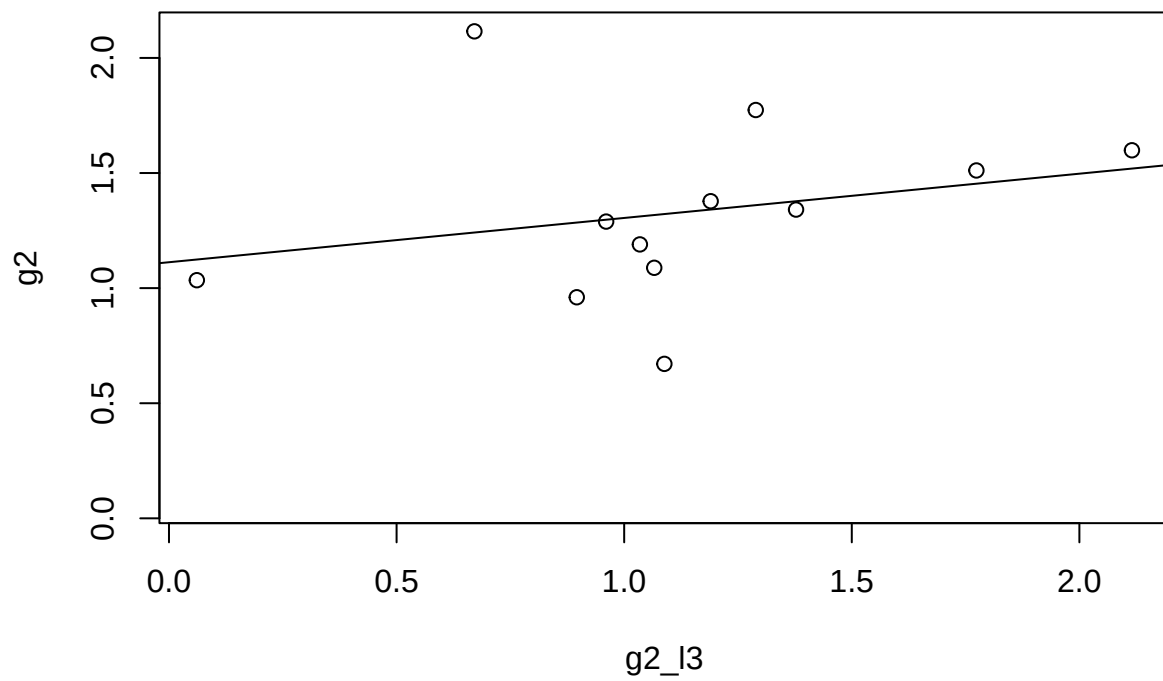
```

plot(g2_l2, g2); abline(lm(g2 ~ g2_l2))

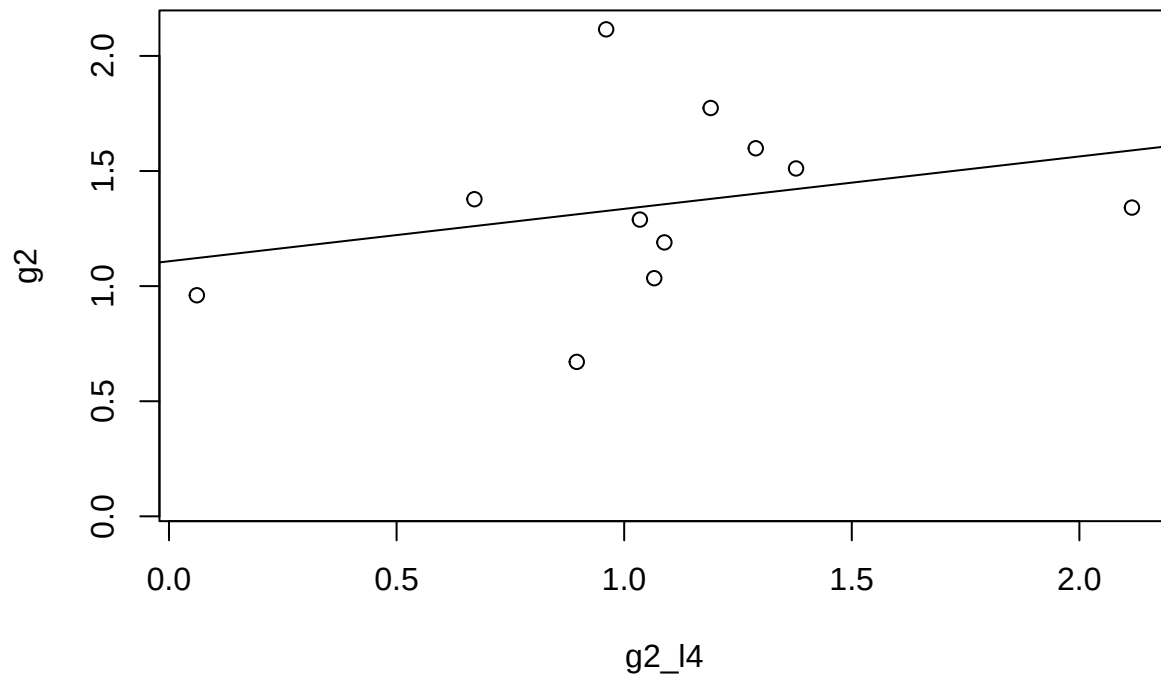
```



```
plot(g2_l3, g2); abline(lm(g2 ~ g2_l3))
```



```
plot(g2_l3, g2); abline(lm(g2 ~ g2_l3))
```



Interpretation: Quarterly GDP growth has a slight autocorrelation with lags, most notably the lag at  $k=1$ . This means that behaviors and production adjusts primarily to reflect the previous quarter. Quarters before  $t=-1$  are also a factor, but not by as much.

#Textbook B

### 2.10.3

a)

```
tute1 <- read_csv("tute1.csv")
```

```
## Rows: 100 Columns: 4
## -- Column specification -----
## Delimiter: ","
## dbl (3): Sales, AdBudget, GDP
## date (1): Quarter
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

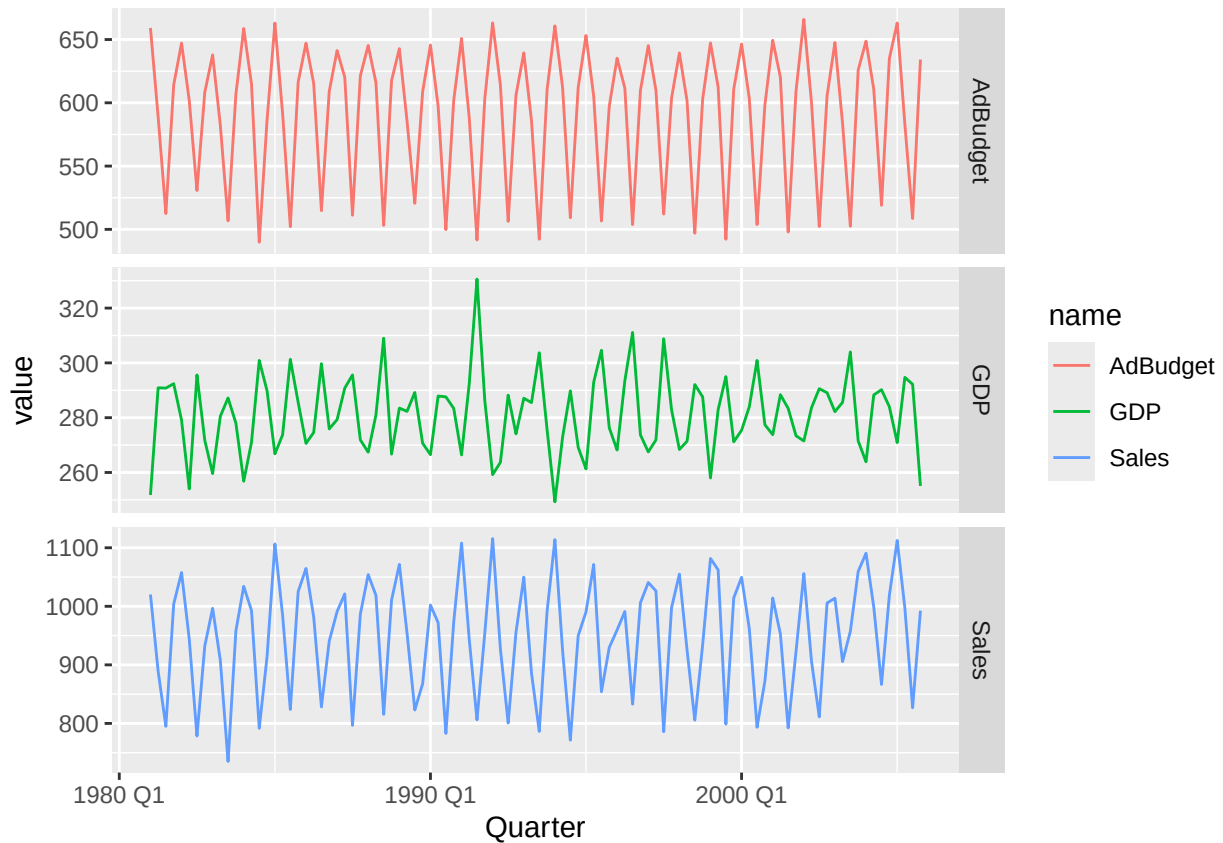
```
#view(tute1)
```

b)

```
mytimeseries <- tutel |>
  mutate(Quarter = yearquarter(Quarter)) |>
  as_tsibble(index = Quarter)
```

c)

```
mytimeseries |>
  pivot_longer(-Quarter) |>
  ggplot(aes(x = Quarter, y = value, colour = name)) +
  geom_line() +
  facet_grid(name ~ ., scales = "free_y")
```



When the `facet_grid()` line is removed, the three graphs are combined into one larger plot

## 2.10.4

a)



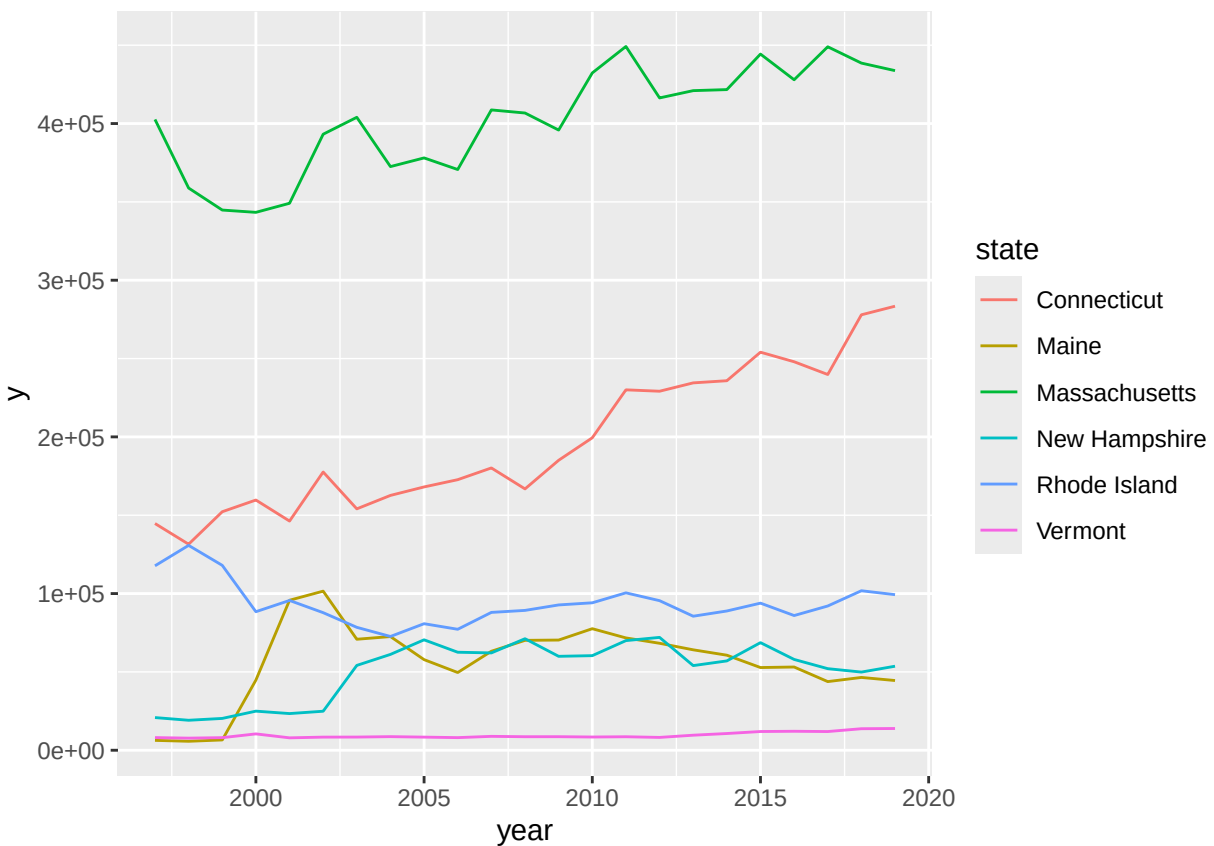
```
#install.packages("USgas")
```

b)

```
gas <- as_tsibble(us_total, index = year, key = state)
```

c)

```
ne <- c("Maine", "Vermont", "New Hampshire", "Massachusetts", "Connecticut", "Rhode Island")  
gas %>%  
  filter(state %in% ne) %>%  
  ggplot(aes(x = year, y = y, color = state, group = state)) +  
  geom_line()
```



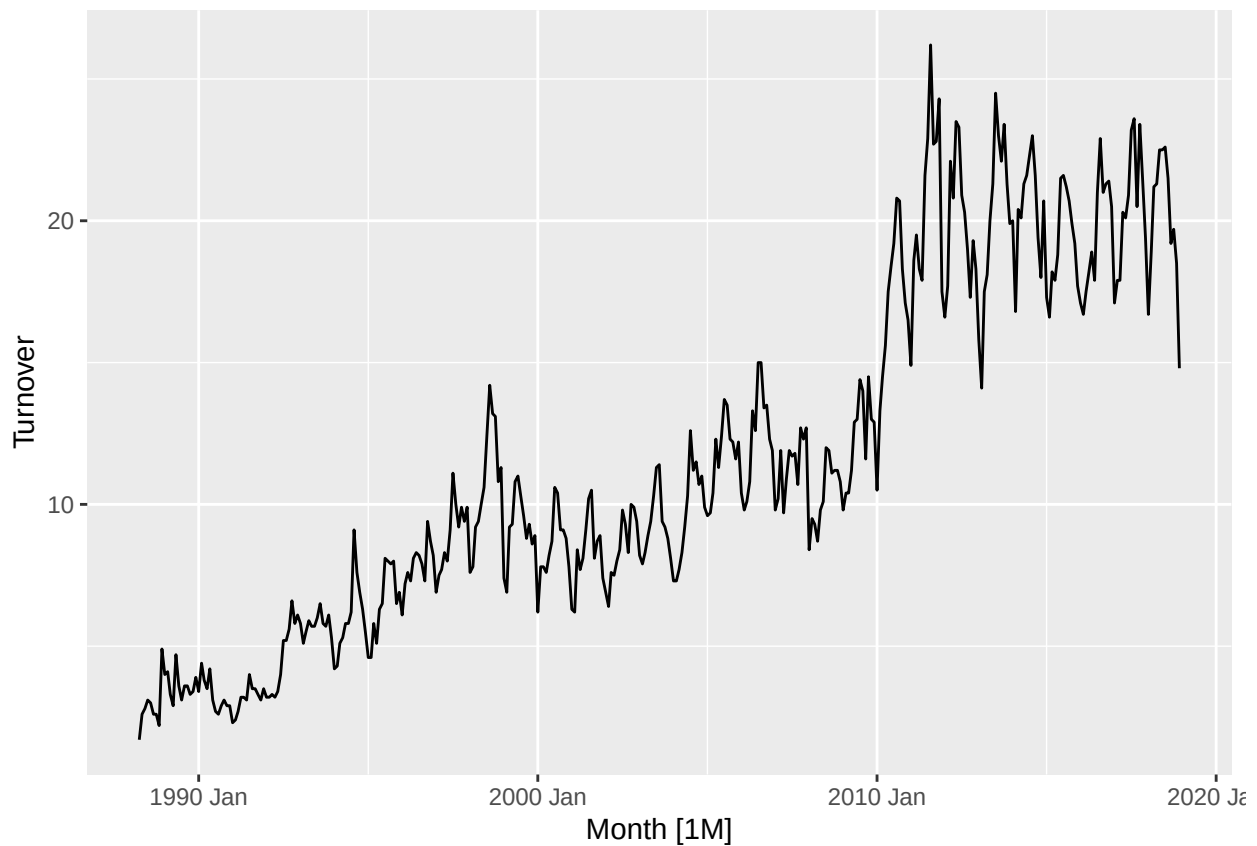
## 2.10.7

```
#install.packages("fpp3")

set.seed(205259899)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`,1))

autoplot(myseries)
```

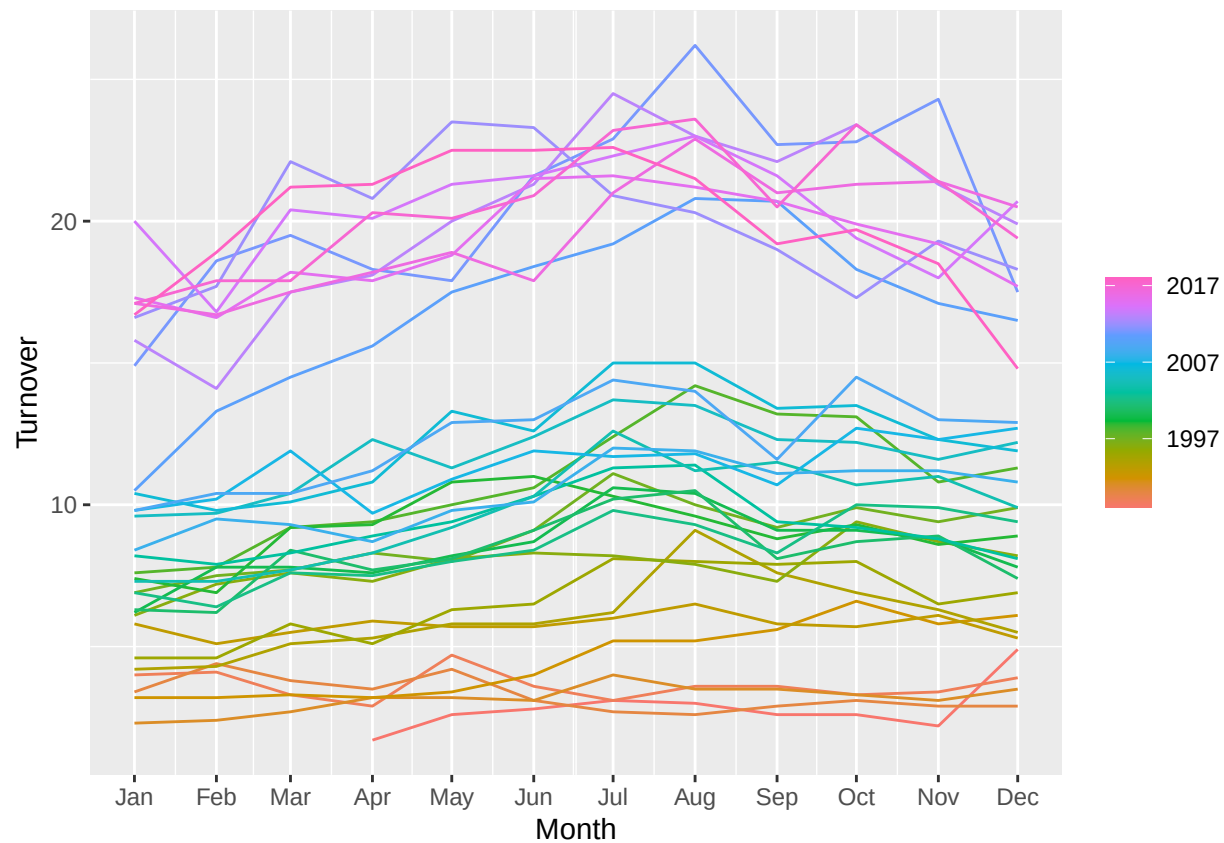
```
## Plot variable not specified, automatically selected '.vars = Turnover'
```



```
gg_season(myseries)
```

```
## Warning: 'gg_season()' was deprecated in feasts 0.4.2.
## i Please use 'ggtime::gg_season()' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

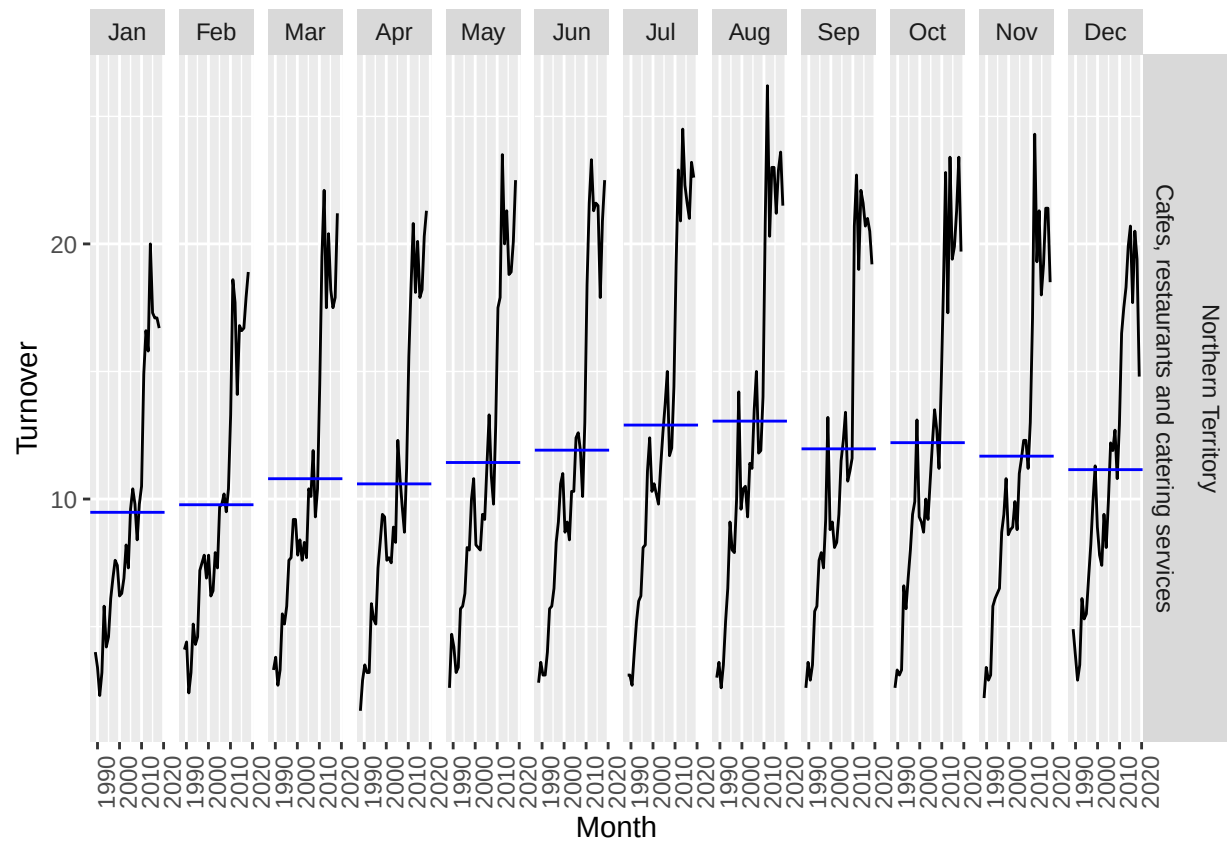
```
## Plot variable not specified, automatically selected 'y = Turnover'
```



```
gg_subseries(myseries)
```

```
## Warning: 'gg_subseries()' was deprecated in feasts 0.4.2.
## i Please use 'ggtime::gg_subseries()' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

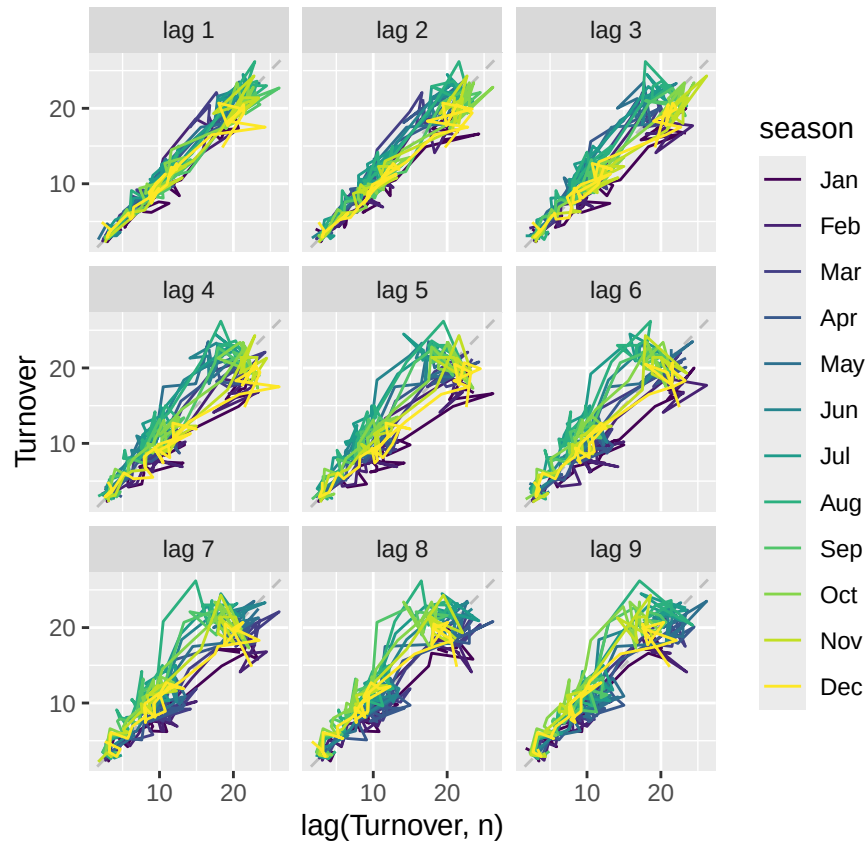
```
## Plot variable not specified, automatically selected 'y = Turnover'
```



```
gg_lag(myseries)
```

```
## Warning: 'gg_lag()' was deprecated in feasts 0.4.2.
## i Please use 'ggtime::gg_lag()' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```
## Plot variable not specified, automatically selected 'y = Turnover'
```



It seems like turnover spiked in Jan 2010, and never returned to previous average levels. There tends to be a turnover spike in July - November compared to the other months. Adding in lags doesn't change the graphs too much, but it seems to add more variance around lag 3.

## 2.10.9

1 = B 2 = A 3 = D 4 = C

## 3.7.1

```
ge <- global_economy |>
  mutate(gdp_pc = GDP / Population) |>
  arrange(desc(gdp_pc))
```

```
## Warning: Current temporal ordering may yield unexpected results.
## i Suggest to sort by 'Country', 'Year' first.
```

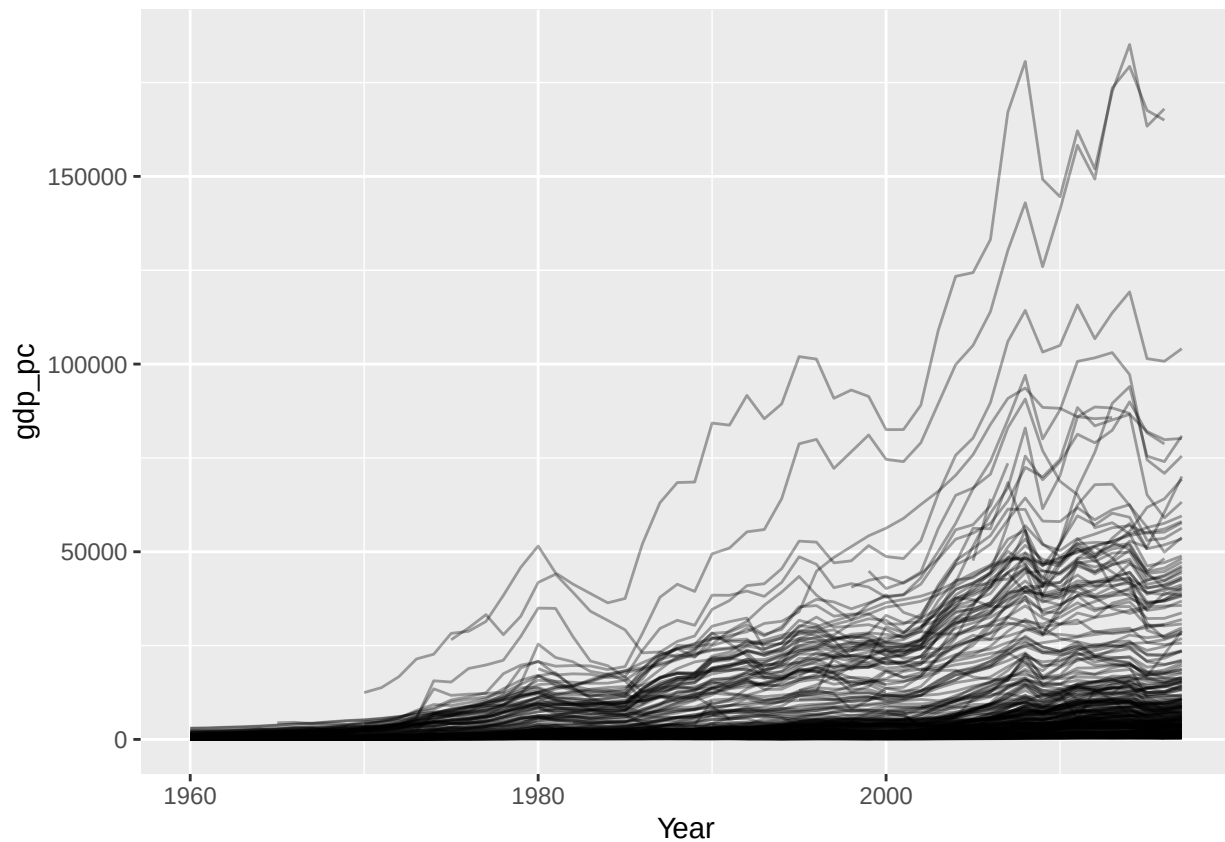
```
head(ge)
```

```
## Warning: Current temporal ordering may yield unexpected results.
## i Suggest to sort by 'Country', 'Year' first.
```

```
## # A tibble: 6 x 10 [1Y]
## # Key:      Country [2]
##   Country    Code  Year  GDP Growth  CPI Imports Exports Population gdp_pc
##   <fct>      <fct> <dbl>  <dbl>  <dbl> <dbl>  <dbl>  <dbl>  <dbl>
## 1 Monaco     MCO   2014  7.06e9  7.18    NA     NA     NA     38132  1.85e5
## 2 Monaco     MCO   2008  6.48e9  0.732   NA     NA     NA     35853  1.81e5
## 3 Liechtenste~ LIE   2014  6.66e9  NA      NA     NA     NA     37127  1.79e5
## 4 Liechtenste~ LIE   2013  6.39e9  NA      NA     NA     NA     36834  1.74e5
## 5 Monaco     MCO   2013  6.55e9  9.57    NA     NA     NA     37971  1.73e5
## 6 Monaco     MCO   2016  6.47e9  3.21    NA     NA     NA     38499  1.68e5
```

```
ge |>
  ggplot(aes(Year, gdp_pc, group = Country)) +
  geom_line(alpha = 0.35)
```

```
## Warning: Removed 3242 rows containing missing values or values outside the scale range
## ('geom_line()').
```



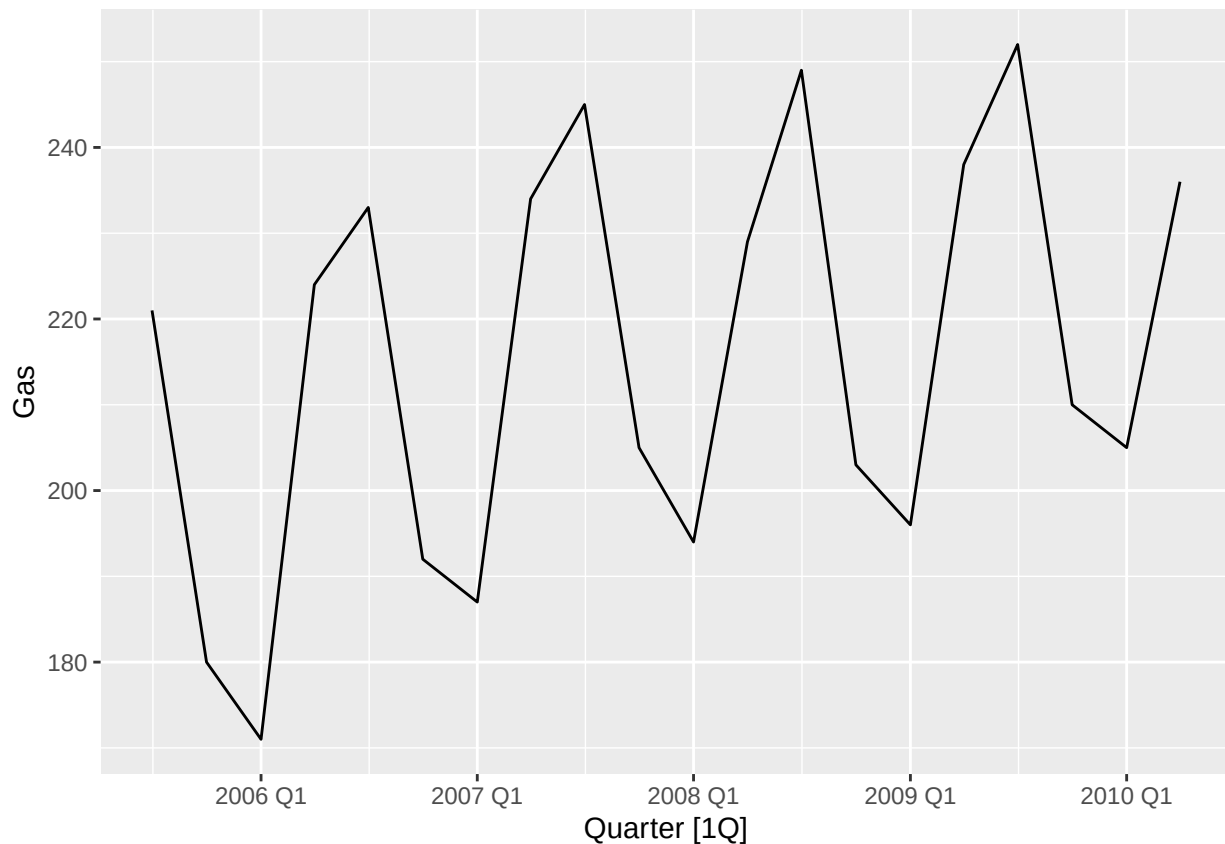
Highest GDP/Cap ever was 2014 Monaco at 185k, but the top country right now is Luxembourg. The top country changes a lot year by year, most recently between Monaco and Luxembourg but historically there have been other frontrunners, like Lichtenstein (according to the data table)

### 3.7.7

```
gas <- tail(aus_production, 5*4) |> dplyr::select(Gas)
```

a)

```
autoplot(gas, Gas)
```



Yes, every Q1, sales bottom out, and every Q3, they peak for the year. The graph has an overall positive trend, with each year improving on the previous year

b)

```
gas_ts <- gas |>
  mutate(Quarter = yearquarter(as.character(Quarter)) |> tsibble::yearquarter()) |>
  mutate(Quarter = tsibble::yearquarter(Quarter)) |> # idempotent; safe
  as_tsibble(index = Quarter)

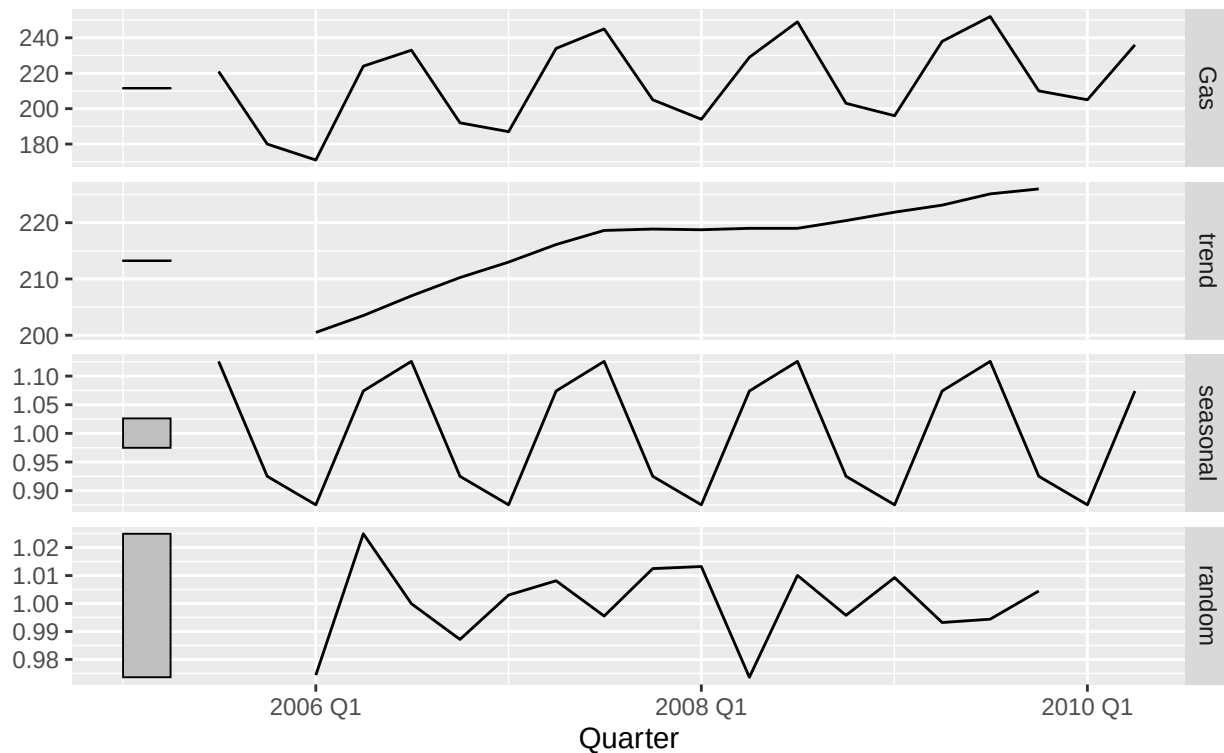
fit <- gas_ts |>
  model(classical_decomposition(Gas, type = "multiplicative"))

cmp <- components(fit)
autoplot(cmp)
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## ('geom_line()').
```

## Classical decomposition

Gas = trend \* seasonal \* random



### c)

The above graphs confirm the data. There is a clear seasonal pattern, and an upwards trend is present. There is some randomness present, which is good because this is a real dataset, but the randomness does not alter the patterns.

d)

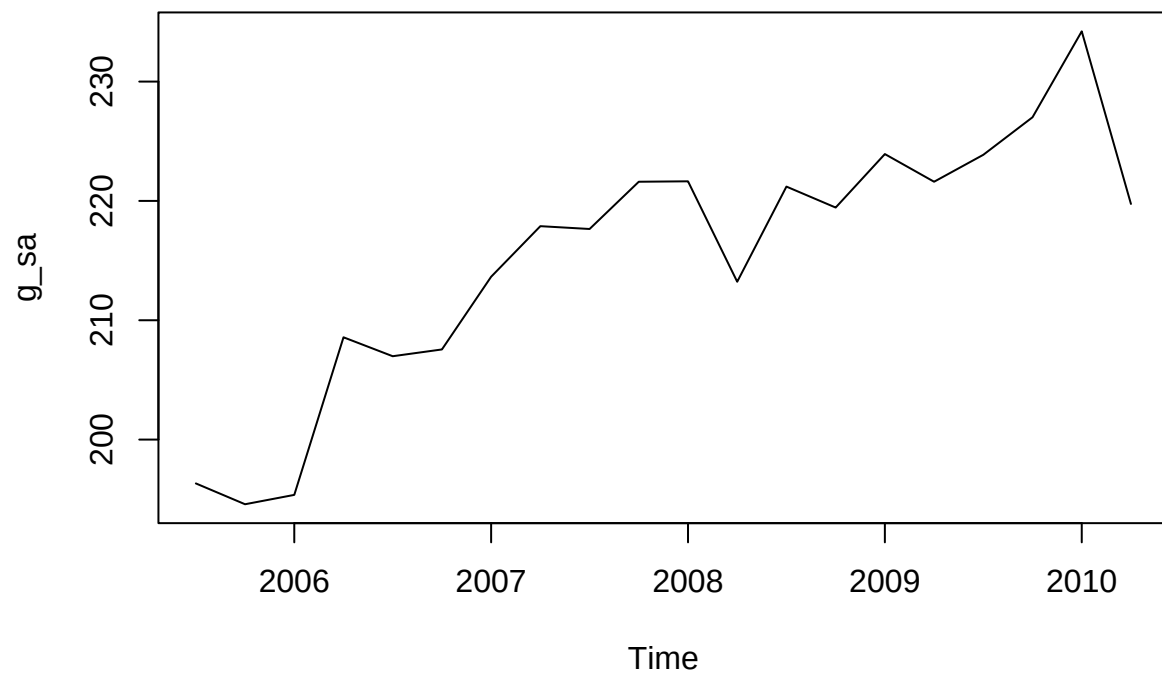
```
yr1 <- as.integer(substr(as.character(gas$Quarter[1]), 1, 4))
q1 <- as.integer(sub(".*Q", "", as.character(gas$Quarter[1])))
g_ts <- ts(gas$Gas, start = c(yr1, q1), frequency = 4)

dc <- decompose(g_ts, type = "multiplicative")

g_sa <- dc$x / dc$seasonal

plot(g_sa, type = "l")
```





e

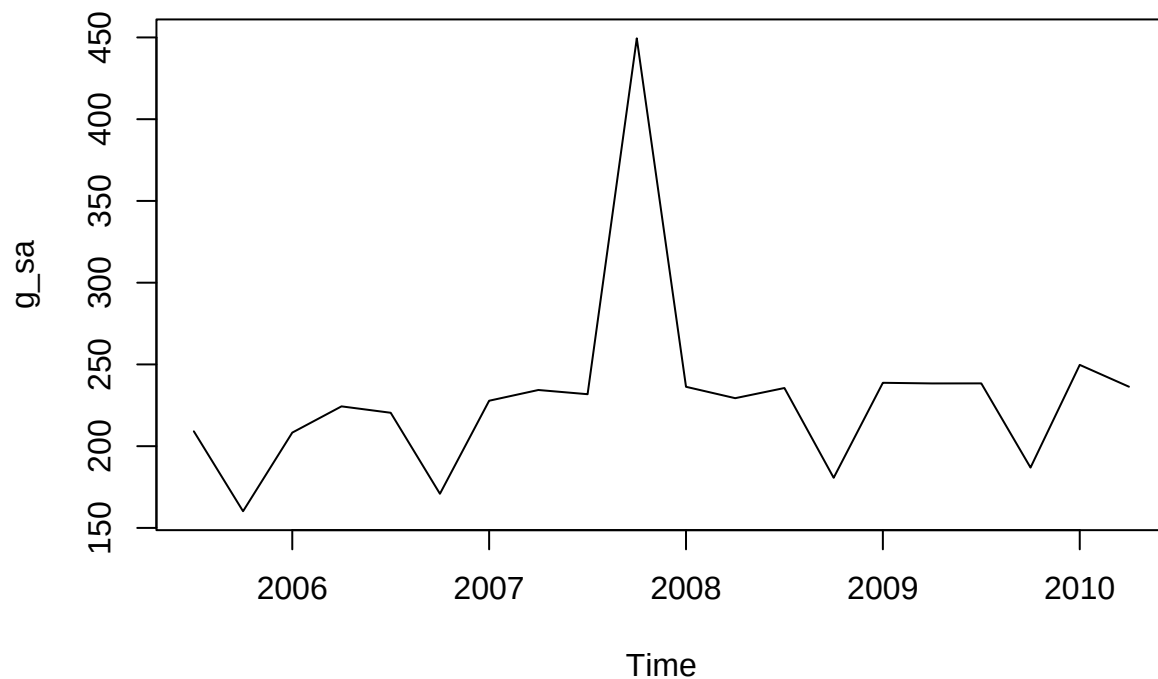
```
gas_outlier <- gas %>%
  mutate(Gas = if_else(row_number() == 10, Gas + 300, Gas))

yr1 <- as.integer(substr(as.character(gas_outlier$Quarter[1]), 1, 4))
q1 <- as.integer(sub(".*Q", "", as.character(gas_outlier$Quarter[1])))
g_ts <- ts(gas_outlier$Gas, start = c(yr1, q1), frequency = 4)

dc <- decompose(g_ts, type = "multiplicative")

g_sa <- dc$x / dc$seasonal

plot(g_sa, type = "l")
```



Adding 300 to a random point (point 10 in this case) severely alters the seasonal graph. Because it is now such a major outlier, the rest of the seasonally-altered data seems constant by comparison.

f)

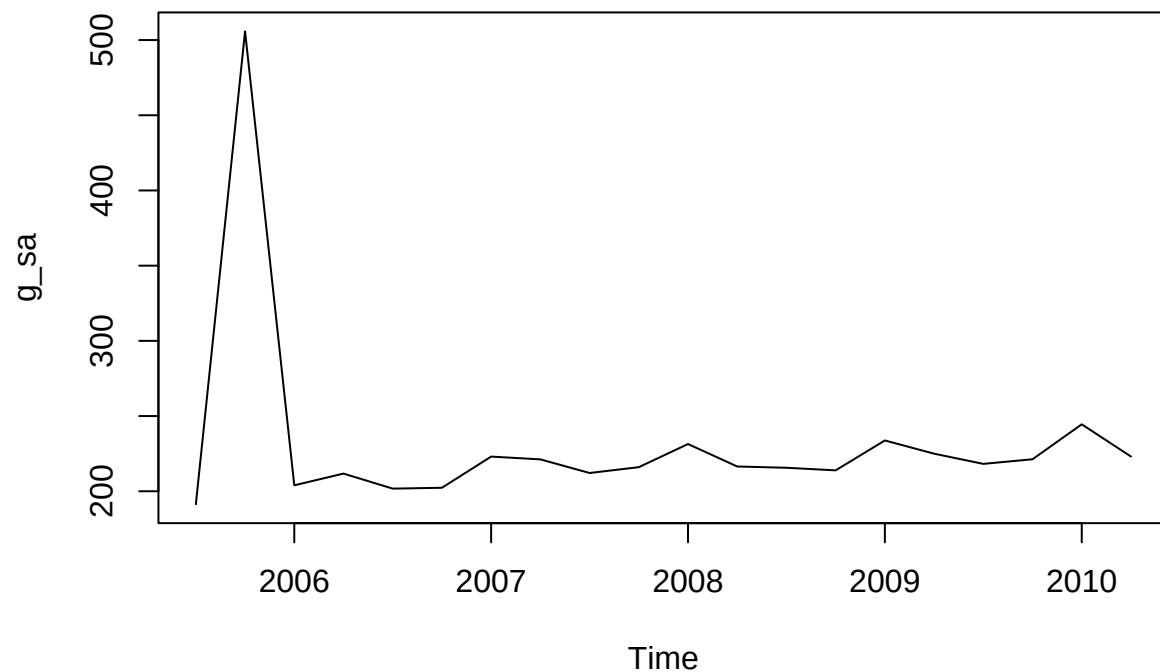
```
gas_outlier_early <- gas %>%
  mutate(Gas = if_else(row_number() == 2, Gas + 300, Gas))

yr1 <- as.integer(substr(as.character(gas_outlier_early$Quarter[1]), 1, 4))
q1 <- as.integer(sub(".*Q", "", as.character(gas_outlier_early$Quarter[1])))
g_ts <- ts(gas_outlier_early$Gas, start = c(yr1, q1), frequency = 4)

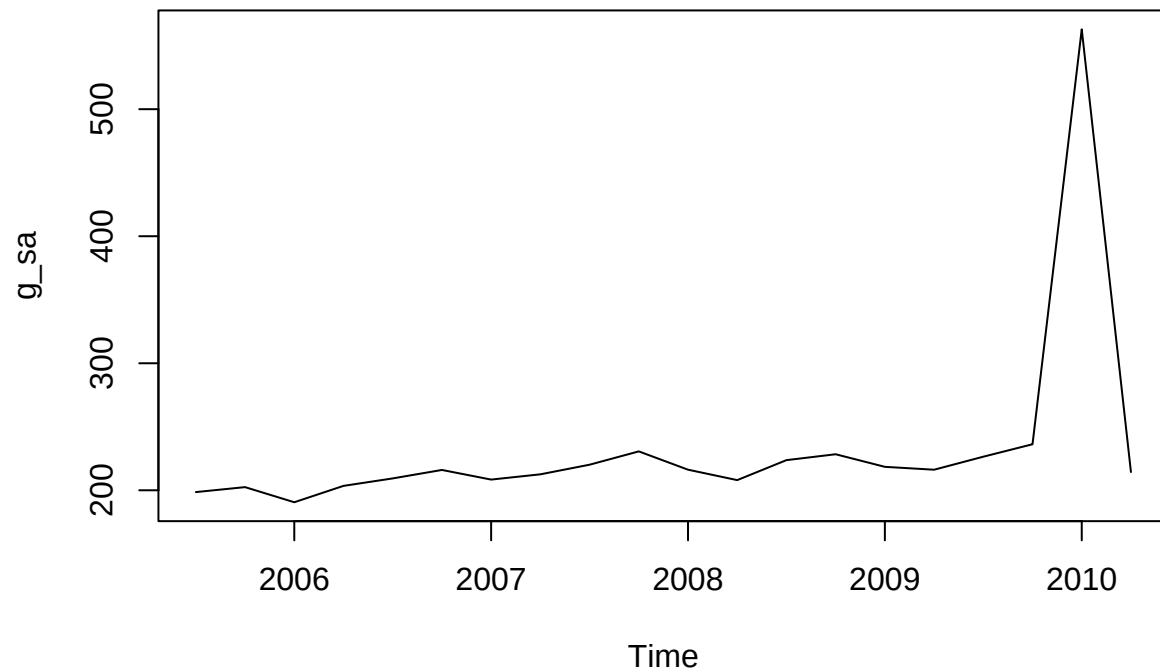
dc <- decompose(g_ts, type = "multiplicative")

g_sa <- dc$x / dc$seasonal

plot(g_sa, type = "l")
```



```
gas_outlier_late <- gas %>%  
  mutate(Gas = if_else(row_number() == 19, Gas + 300, Gas))  
  
yr1 <- as.integer(substr(as.character(gas_outlier_late$Quarter[1]), 1, 4))  
q1  <- as.integer(sub(".*Q", "", as.character(gas_outlier_late$Quarter[1])))  
g_ts <- ts(gas_outlier_late$Gas, start = c(yr1, q1), frequency = 4)  
  
dc <- decompose(g_ts, type = "multiplicative")  
  
g_sa <- dc$x / dc$seasonal  
  
plot(g_sa, type = "l")
```



In both cases, the unaltered data appears to be constant. This is the same affect as in part e.

### 3.7.8

```
## 2.10.7 Code

set.seed(205259899)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`, 1))

ts_turn <- ts(
  myseries$Turnover,
  start = c(lubridate::year(min(myseries$Month)),
            lubridate::month(min(myseries$Month))),
  frequency = 12
)

library(seasonal)

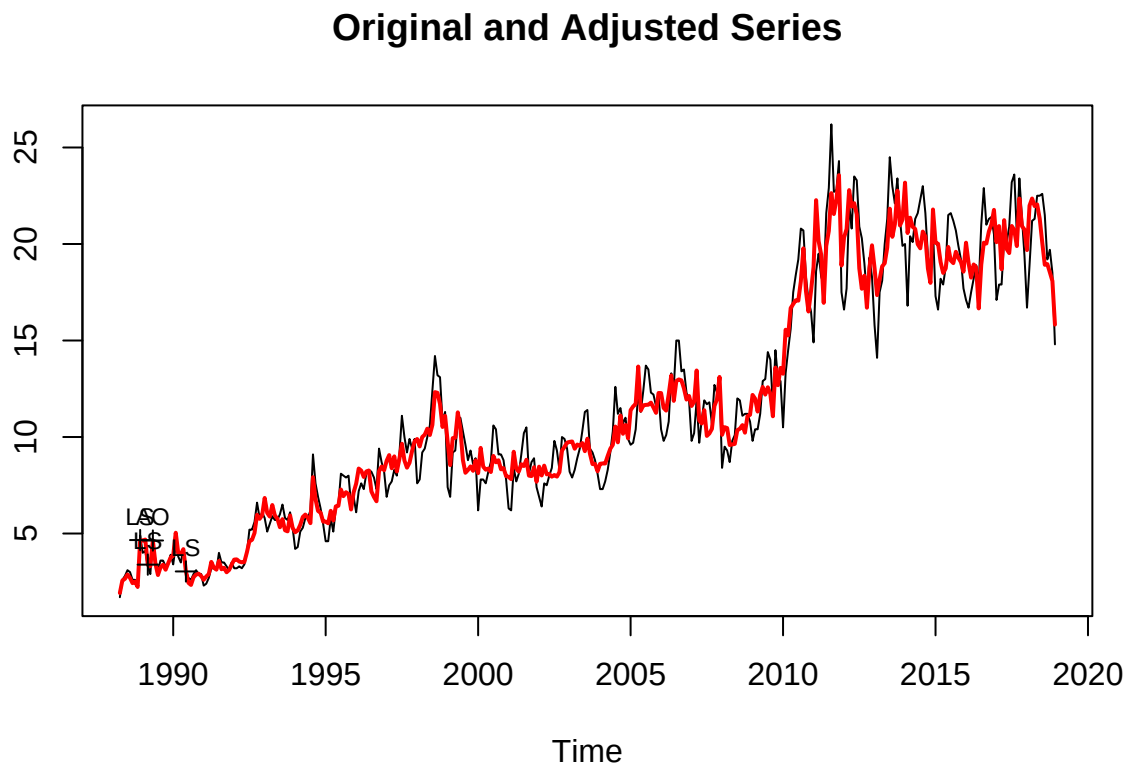
##
## Attaching package: 'seasonal'

## The following object is masked from 'package:tibble':
##
```

```
##      view

## The following objects are masked from 'package:astsa':
##
##      trend, unemp

m <- seas(ts_turn, x11 = "")
plot(m)
```



The X-11 adjustment doesn't seem to reveal too many outliers, but it does smooth the curve and help adjust for season-to-season ambiguity.